

전자전기컴퓨터설계실험II

실험 전 보고서

LAB6: STM32_2 UART 및 FPGA 주변장치

학과 전자전기컴퓨터공학부
이름/학번 김영서 / 2025440031
작성일 2026년 10월 30일
대상 UART 통신, 서보모터, SRAM, DAC
검증 환경
FPGA 시뮬레이터: Icarus Verilog / Vivado Simulator

1. 실험 목적 및 사전 준비 범위

1.1 실험 목적

이번 LAB6의 목적은 PC에서 입력한 한 바이트를 STM32가 수신하고, UART3를 통해 FPGA로 전달한 뒤 FPGA가 수신 문자를 에코하고 LED 명령을 수행하도록 구성하는 것이다. 이어서 FPGA의 PWM, 양방향 메모리 버스, 병렬 DAC 제어를 Verilog로 모듈화하고 자기검사 테스트벤치로 검증한다.

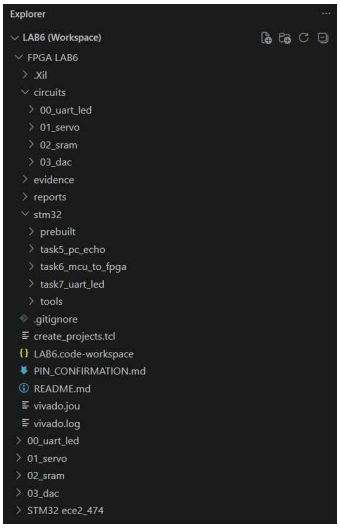


그림 1. LAB6 작업공간과 파일 구성

1.2 공통 조건

항목	조건
MCU	NUCLEO-G474RE, UART2 VCP / UART3 PC10·PC11
UART	9600 baud, 8 data, no parity, 1 stop, newline None
FPGA	HBE-Combo II-DLD Spartan-7 S75, xc7s75fgga484-1
클록·리셋	50 MHz(B6), HIGH reset(K4)
사전 검증	RTL 컴파일, self-checking PASS, VCD 생성
실험일	2026년 10월 26일

표 1. LAB6 공통 실험 조건

2. STM32_2 과제5 · PC와 MCU 입출력

2.1 요구사항과 구현

getchar()의 결과를 int로 받아 EOF를 구분하고, 유효한 문자이면 문자와 uint8_t로 변환한 바이트 값을 16진수 두 자리로 출력한다. 입력 1, A, a의 예상값은 각각 0x31, 0x41, 0x61이다.

입력	ASCII 16진수	예상 출력
1	0x31	받은 문자: 1, 바이트 값: 0x31
A	0x41	받은 문자: A, 바이트 값: 0x41
a	0x61	받은 문자: a, 바이트 값: 0x61

표 2. 과제5 입력별 예상값



그림 2. STM32_2 과제5 문자 및 16진수 출력 코드

2.2 본 실험 확인

MCU VCP COM 포트를 9600 8N1로 열고 줄바꿈을 None으로 설정한다. 세 문자를 각각 한 번씩 보내 문자와 바이트 값이 일치하는지 확인한다. 입력 대기 상태와 오류 정지를 구분한다.

3. STM32_2 과제6·7 및 FPGA UART LED

3.1 데이터 경로



그림 3. PC–STM32–FPGA–PC UART 연결 구조

과제6은 PC→UART2 VCP→STM32→UART3 PC10→FPGA 확장 GPIO RX→FPGA USB-UART F6→PC의 흐름이다. STM32 송신 함수의 반환값 0은 함수 성공만 의미하므로 FPGA 포트에서 같은 문자가 출력되는지 별도로 확인한다. 양쪽 보드의 GND를 반드시 공통으로 연결한다.



그림 4. STM32_2 과제6 UART3 전송 코드



그림 5. STM32_2 과제7 LED 명령 전송 코드

3.2 LED 명령

문자	ASCII	LED 동작
0	0x30	끄기
1	0x31	켜기
2	0x32	현재 상태 반전
그 밖	임의	상태 유지

표 3. 과제7 UART LED 명령표

수신 유효 펄스에서 명령을 해석하고 같은 바이트를 UART TX로 에코한다. 1을 반복하면 계속 켜지고, 2를 반복하면 매번 반전되어야 한다. 리셋 시 LED는 꺼지고 TX는 HIGH 대기 상태다.

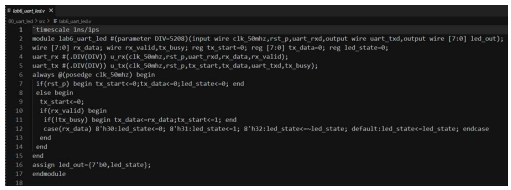


그림 6. FPGA UART 수신·에코·LED 명령 처리 코드

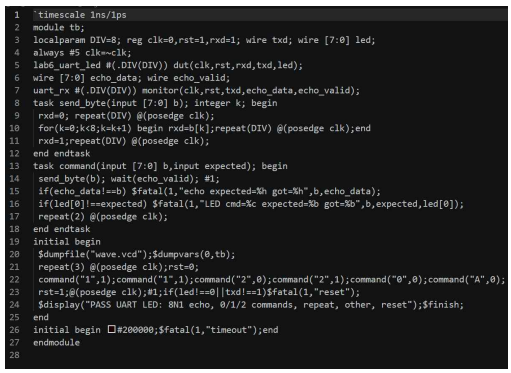


그림 7. UART LED 자기검사 테스트벤치



그림 8. UART LED 자기검사 PASS 결과



그림 9. UART 수신·에코·LED 상태 파형

3.3 XDC와 실제 연결

USB-UART 자체시험 XDC는 RX C6, TX F6, LED0 N5를 사용한다. 과제6·7에서는 RX C6를 그대로 쓰지 않고, 교재에서 선택한 확장 GPIO의 FPGA 패키지 핀으로 바꾼다. MCU PC10은 FPGA 패키지 핀 번호가 아니므로 XDC에 PC10이라고 적지 않는다.

4. FPGA 서보모터 제어

4.1 회로 구조와 예상값

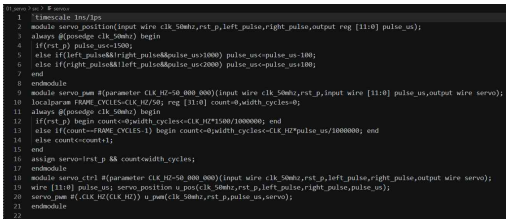


그림 10. 서보 위치값과 20 ms PWM 생성 구조

servo_position은 중앙 1500 µs에서 시작해 left/right 단일 펄스마다 100 µs씩 감소 또는 증가한다. 범위는 1000~2000 µs이며 두 입력이 동시에 들어오면 상태를 유지한다. servo_pwm은 50 Hz, 즉 20 ms 프레임을 만들고 다음 프레임 경계에서 새 펄스 폭을 적용한다.

상태	HIGH 폭	20 ms 대비 듀티
최소	1.0 ms	5.0%
중앙	1.5 ms	7.5%
최대	2.0 ms	10.0%

표 4. 서보 PWM 예상값



```
`timescale 1ns/1ps
module servo_position(input wire clk_50mhz,rst_p,left_pulse,right_pulse,output reg [11:0] pulse_us);
always @(posedge clk_50mhz) begin
 if(rst_p) pulse_us<=1500;
 else if(left_pulse&&!right_pulse&&pulse_us>1000) pulse_us<=pulse_us-100;
 else if(right_pulse&&!left_pulse&&pulse_us<2000) pulse_us<=pulse_us+100;
end
endmodule
module servo_pwm #(parameter CLK_HZ=50_000_000)(input wire clk_50mhz,rst_p,input wire [11:0] pulse_us,output wire servo);
localparam FRAME_CYCLES=CLK_HZ/50; reg [31:0] count=0,width_cycles=0;
always @(posedge clk_50mhz) begin
 if(rst_p) begin count<=0;width_cycles<=CLK_HZ*1500/1000000; end
 else if(count==FRAME_CYCLES-1) begin count<=0;width_cycles<=CLK_HZ*pulse_us/1000000; end
 else count<=count+1;
end
assign servo=!rst_p && count<width_cycles;
endmodule
module servo_ctrl #(parameter CLK_HZ=50_000_000)(input wire clk_50mhz,rst_p,left_pulse,right_pulse,output wire servo);
wire [11:0] pulse_us; servo_position u_pos(clk_50mhz,rst_p,left_pulse,right_pulse,pulse_us);
servo_pwm #(.CLK_HZ(CLK_HZ)) u_pwm(clk_50mhz,rst_p,pulse_us,servo);
endmodule
```

그림 11.서보 위치·PWM·최상위 설계 코드



그림 12. 서보 자기검사 PASS 결과



그림 13. 1000·1500·2000 µs 서보 파형

4.2 실제 보드 확인

전원과 접지를 교재 방식으로 연결하고 신호선만 FPGA 서보 출력 핀에 연결한다. 서보모터 전류를 FPGA I/O 핀에서 공급하지 않는다. 중앙·좌측 한계·우측 한계의 이동과 상·하한 고정을 영상으로 확인한다.

5. FPGA SRAM 읽기·쓰기

5.1 버스 제어

쓰기 요청만 활성화되면 FPGA가 하위 4비트를 데이터 버스에 구동하고 CS와 WE를 LOW로 만든다. 읽기 요청만 활성화되면 FPGA 출력은 Z가 되고 SRAM이 버스를 구동하며 하위 4비트를 LED에 표시한다. 읽기와 쓰기가 동시에 요청되면 세 제어신호를 비활성 상태로 두어 버스 충돌을 막는다.

ctrl_w	ctrl_r	동작	CS/OE/WE
0	0	대기, FPGA bus=Z	1/1/1
1	0	쓰기, FPGA가 bus 구동	0/1/0
0	1	읽기, FPGA bus=Z	0/0/1
1	1	충돌 차단	1/1/1

표 5. SRAM 제어 진리표



그림 14. SRAM 버스 제어 및 상위 모듈 코드



그림 15. SRAM 자기검사 PASS 결과

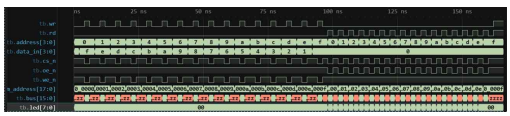


그림 16. SRAM 16주소 쓰기·읽기·Z 버스 파형

5.2 실제 보드 확인

주소 0~15에 서로 다른 값을 쓴 뒤 같은 주소를 읽어 LED 값과 비교한다. 쓰기 중 OE 비활성, 읽기 중 WE 비활성, 대기 중 데이터 버스 Z를 확인한다. SRAM 전용 핀은 교재 표에 따라 모두 지정해야 비트스트림 생성이 가능하다.

6. FPGA DAC

6.1 램프와 병렬 쓰기

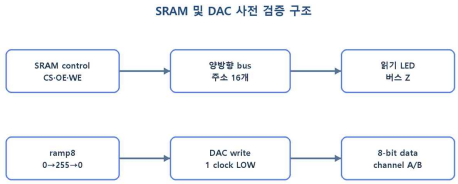


그림 17. SRAM 버스와 DAC 병렬 출력의 사전 검증 구조

ramp8은 설정된 주기마다 8비트 값을 1씩 증가시키며 255 다음에는 0으로 돌아간다. dac_parallel8은 update 시 데이터와 채널을 등록하고 WE를 한 클록 동안 LOW로 만든다. 데이터와 채널은 쓰기 펄스가 끝날 때까지 유지된다.

항목	예상 동작
데이터	0→1→…→255→0
CS_n / LDAC_n	연속 LOW
WE_n	update마다 1클록 LOW
A/B	add_sel 값을 쓰기 시점에 등록
LED	dac_d와 동일

표 6. DAC 디지털 인터페이스 예상값



그림 18. DAC 램프·병렬 쓰기·최상위 코드

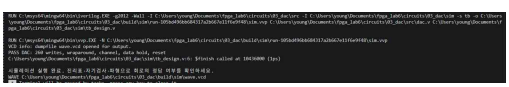


그림 19. DAC 자기검사 PASS 결과

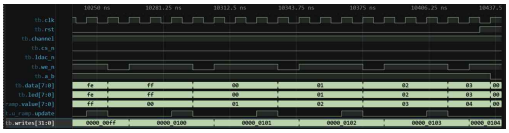


그림 20. DAC FE→FF→00 랩어라운드 파형

6.2 실제 보드 확인

교재 핀표로 DAC 제어선과 D[7:0]를 지정하고 아날로그 출력은 멀티미터 또는 오실로스코프로 측정한다. 디지털 램프가 증가할수록 출력 전압이 단계적으로 증가하고 255 다음 0에서 되돌아가는지 확인한다.

7. 본 실험 진행 계획

순서	예정 작업	확인할 내용
1	장비 교재 핀표 확인	확장 GPIO·서보·SRAM·DAC 실제 핀
2	XDC 작성 및 검토	모든 최상위 포트와 I/O 규격
3	기능 시뮬레이션 재확인	예비 단계 PASS·파형과 비교
4	합성·구현	오류와 타이밍 위반 여부
5	비트스트림 생성·다운로드	대상 FPGA와 파일 확인
6	보드 시연	UART·서보·SRAM·DAC 실제 동작
7	결과 증빙	사진·영상·계측값 확보

표 7. LAB6 본 실험 예정 절차

본 실험에서는 장비 교재의 핀 배치표를 먼저 확인하여 XDC를 작성하고, 기능 시뮬레이션 결과를 재확인한 뒤 합성, 구현, 비트스트림 생성 및 실제 보드 시연을 순서대로 수행할 예정이다. 예비 단계에서는 실제 Vivado 실행 결과를 보고서 증빙으로 사용하지 않는다.

8. 사전 검증 결과 및 체크리스트

회로	self-checking 결과	파형
UART LED	PASS: 에코·0/1/2·반복·기타·리셋	wave.vcd 생성
서보	PASS: 20 ms·1/1.5/2 ms·한계·동시 입력	wave.vcd 생성
SRAM	PASS: 16주소·Z·동시요청	wave.vcd 생성
DAC	PASS: 260회·255→0·채널·hold	wave.vcd 생성

표 8. LAB6 FPGA 사전 시뮬레이션 결과

실험 전 준비 완료 항목은 STM32 과제5~7 독립 코드, FPGA 네 회로 RTL, 자기검사 테스트벤치, VS Code 실행 작업과 장비 핀 확인표다. 실제 XDC 완성, 장치 연결, 합성·구현과 계측 결과는 본 실험 후 결과보고서에 기록한다.

LAB6 실험 결과 및 분석

9. 포트 및 핀 번호 입력

예비보고서의 TODO 핀은 교재의 HBE-Combo II-DLD 핀 구성표와 실제 배선 결과를 기준으로 확정하였다. STM32와 FPGA 사이의 UART 신호는 STM32의 TX와 FPGA의 RX를 연결했다. 또한, 두 장비의 GND를 공통으로 연결하여 설계했다.

- **UART 연동** / 주요 핀: STM32 TX/D1 – FPGA RX, GND 공통 / 기능: 문자 명령 전달
- **Servo** / 주요 핀: B6, K4, N8, N4, AA22 / 기능: CLK, reset, left, right, PWM
- **SRAM** / 주요 핀: K4/N8, A[17:0], D[15:0], N20/L21/U17 / 기능: 쓰기·읽기 및 제어
- **DAC** / 주요 핀: B6, K4, Y1, E1/F4/G4/G2, D[7:0] / 기능: 카운트 데이터 출력

10. 실험 과정 및 결과

본 실험에서는 STM32 NUCLEO-G474RE의 UART 송수신 루틴을 구성한 이후, FPGA에서는 UART LED 제어와 서보 PWM, 비동기 SRAM 인터페이스, 병렬 DAC 제어 회로를 Verilog를 통해서 설계하여 진행하였다.

각 회로는 자기검사 테스트벤치를 거쳐 기능을 검증한 뒤 합성, 구현, 비트스트림 생성을 vivado에서 진행하였다. 또한, 실제 장비 동작을 확인하였다.

검증 결론

STM32 과제 5는 PC-UART 에코를 통해 확인하였고, 과제 6은 STM32 송신 데이터를 별도 시리얼 모니터에서 수신하는 과정을, 과제 7은 STM32와 FPGA를 연결하여 문자 명령과 LED 상태를 확인하는 과정을 영상으로 검증하고 확인하였다. FPGA 네 회로는 모두 vivado 내에서 시뮬레이션 결과 모두 PASS를 얻었으며 UART LED, Servo, DAC는 타이밍 제약을 만족한 상태로 비트스트림을 생성하였다. SRAM은 클록이 없는 조합회로이므로 별도의 사용자 타이밍 제약 없이 비트스트림 생성 완료를 확인하였다.

- **STM32 과제 5** / 핵심 검증: PC 입력 문자 에코 / 결과: 영상 확인
- **STM32 과제 6** / 핵심 검증: UART 송신 및 수신 모니터 비교 / 결과: 영상 확인
- **STM32 과제 7** / 핵심 검증: STM32-FPGA UART LED 연동 / 결과: 영상 확인
- **UART LED** / 핵심 검증: 8N1 수신·에코·0/1/2 명령 / 결과: PASS / Bitstream
- **Servo** / 핵심 검증: 20 ms 프레임, 1/1.5/2 ms / 결과: PASS / Bitstream
- **SRAM** / 핵심 검증: 16주소 쓰기·읽기·버스 해제 / 결과: PASS / Bitstream
- **DAC** / 핵심 검증: 260회 쓰기·wraparound·채널 / 결과: PASS / Bitstream

10.1 STM32 UART 과제

10.1.1 공통 구현 방법

STM32Cube 기반 프로젝트에서 사용자 루틴 파일을 과제별로 교체하고 USART를 9600 baud, 8 data bit, no parity, 1 stop bit 조건으로 사용하였다. 전송 성공 여부는 HAL_UART_Transmit의 반환값 HAL_OK, 즉 정수 0으로 판단하였다. 한글 메시지는 시리얼 모니터 문자 인코딩 차이로 깨질 수 있으므로 결과 판정은 영문 문자, 16진수 바이트 값, 반환값을 기준으로 하여 진행하였다.

```
#include "ece2 474.h"
#include "main.h"
#include <stdint.h>
#include <stdio.h>

void ece2 474 init(void) {}
void polling routine(void){int
ch=getchar();if(ch!=EOF){uint8 t
data=(uint8 t)ch;HAL StatusTypeDef
result=HAL UART Transmit(&huart3,&data,1U,100U);p
rintf("TX char: %c, byte: 0x%02X, result:
%d\r\n",ch,(unsigned)data,(int)result);}}

static void tim6 isr p4(void) {}
static void uart3 isr p5(void) {}
static void uart2 isr p6(void) {}
static void button isr p7(void) {}
static void gpio in pullup isr p8 ece2(void) {}
static void gpio in pulldown isr p9 ece2(void) {}
void HAL GPIO EXTI Callback(uint16 t
GPIO Pin){if(GPIO Pin==BUTTON ece2 Pin)button isr
 p7();else
if(GPIO Pin==GPIO IN PULLUP ece2 Pin)gpio in pull
up isr p8 ece2();else
if(GPIO Pin==GPIO IN PULLDOWN ece2 Pin)gpio in pu
lldown isr p9 ece2();}
void
HAL TIM PeriodElapsedCallback(TIM HandleTypeDef
*htim){if(htim->Instance==TIM6)tim6 isr p4();}
void HAL UART RxCpltCallback(UART HandleTypeDef
*huart){if(huart->Instance==USART2)uart2 isr p6()
;else if(huart->Instance==USART3)uart3_isr_p5();}
```

10.1.2 과제 5 PC UART 에코

PC에서 보낸 문자를 STM32가 수신한 뒤 다시 PC로 반환하도록 구성하였다. 영상에서 Serial Monitor에 Hello World 출력이 반복되고 문자 1을 보냈을 때 Received: 1, byte: 0x31이 표시되는 것을 확인하였고, 이를 통해 입력 문자와 수신 바이트가 일치하다는 것을 알 수 있었다..



그림 1 과제 5 STM32 빌드 및 실행 자료

10.1.3 과제 6 STM32 송신

STM32가 USART3를 통해 1, 2, A 등의 문자를 송신하도록 구현하였다. 영상에서는 왼쪽 시리얼 모니터에 TX char, byte, result가 표시되고 별도의 오른쪽 모니터에서 수신 문자가 누적되는 모습을 확인하였다. result 0은 HAL_UART_Transmit 호출이 성공했음을 의미하며, 송신 데이터와 수신 화면의 대응으로 UART 경로를 검증하였다.



그림 2 과제 6 UART 송신 코드 및 결과

10.1.4 과제 7 STM32 FPGA 연동

STM32 TX와 FPGA UART RX를 연결하고 두 장비의 GND를 공통으로 구성하였다. PC에서 입력한 0, 1, 2, A 등의 문자가 STM32에서 송신되며, 영상에서는 노트북의 송신 기록과 FPGA 보드의 LED 상태를 동시에 확인하였다. UART 함수의 반환값은 모두 0이었고, 실제 보드 연동은 영상 증거를 기준으로 판정하였다.



그림 3 과제 7 STM32 FPGA 연동 자료

10.2 FPGA UART LED 회로

10.2.1 설계 및 시뮬레이션

UART 수신기는 start bit, 8 data bit, stop bit 순으로 직렬 데이터를 복원하고 rx_valid를 한 클록 동안 발생시킨다. 수신 바이트는 송신기로 다시 에코하며, ASCII 0은 LED를 끄고 1은 켜며 2는 현재 상태를 반전한다. 그 외 문자는 LED 상태를 유지한다.

```
`timescale 1ns/1ps
module lab6_uart_led #(parameter DIV=5208)(input
wire clk 50mhz,rst p,uart rxd,output wire
uart txd,output wire [7:0] led out);
wire [7:0] rx data; wire rx valid,tx busy; reg
tx start=0; reg [7:0] tx data=0; reg led state=0;
uart rx #(.DIV(DIV))
u rx(clk 50mhz,rst p,uart rxd,rx data,rx valid);
uart tx #(.DIV(DIV))
u tx(clk 50mhz,rst p,tx start,tx data,uart txd,tx
 busy);
always @(posedge clk 50mhz) begin
 if(rst p) begin
tx start<=0;tx data<=0;led state<=0; end
 else begin
  tx start<=0;
  if(rx valid) begin
   if(!tx busy) begin
tx data<=rx data;tx start<=1; end
   case(rx data) 8'h30:led state<=0;
8'h31:led state<=1; 8'h32:led state<=~led state;
default:led state<=led state; endcase
  end
 end
end
assign led out={7'b0,led state};
endmodule
```



그림 4 UART 수신 데이터와 LED 상태 파형

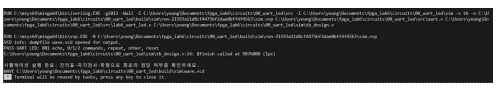


그림 5 UART LED 자기검사 PASS

10.2.2 구현 결과

Vivado 구현 결과 WNS 14.587 ns, TNS 0.000 ns, WHS 0.156 ns, THS 0.000 ns였으며 실패 endpoint와 실패 route는 0이었다. 모든 사용자 타이밍 제약을 만족한 상태로 비트스트림 생성을 완료하였다.



그림 6 UART LED 합성 및 비트스트림 완료

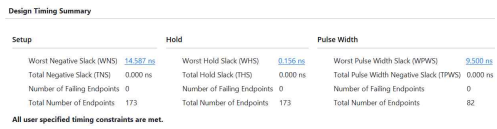


그림 7 UART LED 타이밍 요약

10.3 FPGA 서보 PWM 회로

10.3.1 설계 및 시뮬레이션

서보 위치 레지스터는 리셋 시 1500 us로 초기화된다. 왼쪽 입력은 1000 us까지 100 us 단위로 감소시키고 오른쪽 입력은 2000 us까지 증가시킨다. PWM 블록은 50 Hz, 즉 20 ms 프레임을 만들고 pulse_us에 대응하는 구간만 servo 출력을 High로 유지한다.

```
`timescale 1ns/1ps
module servo position(input wire
clk 50mhz,rst p,left pulse,right pulse,output reg
[11:0] pulse us);
always @(posedge clk 50mhz) begin
 if(rst p) pulse us<=1500;
 else if(left pulse&&!right pulse&&pulse us>1000)
pulse us<=pulse us-100;
 else if(right pulse&&!left pulse&&pulse us<2000)
pulse us<=pulse us+100;
end
endmodule
module servo pwm #(parameter
CLK HZ=50 000 000)(input wire
clk 50mhz,rst p,input wire [11:0] pulse us,output
wire servo);
localparam FRAME CYCLES=CLK HZ/50; reg [31:0]
count=0,width cycles=0;
always @(posedge clk 50mhz) begin
 if(rst p) begin
count<=0;width cycles<=CLK HZ*1500/1000000; end
 else if(count==FRAME CYCLES-1) begin
count<=0;width cycles<=CLK HZ*pulse us/1000000;
end
 else count<=count+1;
end
assign servo=!rst p && count<width cycles;
endmodule
module servo ctrl #(parameter
CLK HZ=50 000 000)(input wire
clk 50mhz,rst p,left pulse,right pulse,output
wire servo);
wire [11:0] pulse us; servo position
u pos(clk 50mhz,rst p,left pulse,right pulse,puls
e us);
servo pwm #(.CLK HZ(CLK HZ))
u pwm(clk 50mhz,rst p,pulse us,servo);
endmodule
```



그림 8 서보 20 ms 프레임 및 펄스 폭 파형



그림 9 서보 자기검사 PASS

10.3.2 실제 장비 결과

서보_PWM_동작1.mp4와 서보_PWM_동작2.mp4에서 보드의 입력 버튼을 조작할 때 서보 혼의 방향이 서로 다른 위치로 이동하는 것을 확인하였다. 오실로스코프 측정은 수행하지 않았으므로 실제 펄스 폭 수치는 제시하지 않고, 테스트벤치의 1/1.5/2 ms 검증과 실제 위치 변화로 결과를 구분하였다.

초기 비트스트림 생성 시 servo 출력의 LOC와 IOSTANDARD가 누락되어 NSTD-1 및 UCIO-1 DRC 오류가 발생하였다. XDC에 PACKAGE_PIN AA22와 IOSTANDARD LVCMOS33을 추가한 뒤 구현과 비트스트림 생성이 완료되었다.

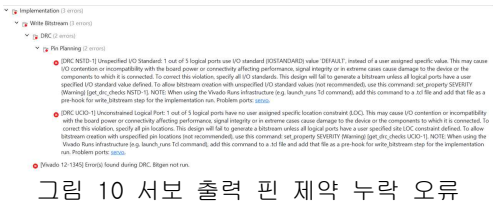


그림 10 서보 출력 핀 제약 누락 오류



그림 11 서보 비트스트림 생성 완료

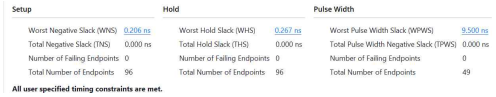


그림 12 서보 타이밍 제약 충족

10.4 FPGA SRAM 인터페이스

10.4.1 설계 및 시뮬레이션

쓰기 요청만 활성화되면 FPGA가 하위 4비트 데이터를 SRAM 데이터 버스에 구동하고, 읽기 요청만 활성화되면 버스를 high impedance로 해제하여 SRAM 출력을 수신한다. 두 요청이 동시에 들어오는 경우에는 읽기와 쓰기를 모두 비활성화하여 버스 충돌을 방지하였다.

```
`timescale 1ns/1ps
module sram_bus_control(input wire
ctrl_w,ctrl_r,output wire
drive_data,read_data,ram_cs,ram_oe,ram_we);
assign drive_data=ctrl_w&&!ctrl_r;assign
read_data=ctrl_r&&!ctrl_w;
assign ram_cs=!(drive_data||read_data);assign
ram_oe=!read_data;assign
ram_we=!drive_data;endmodule
module sram(input wire ctrl_w,ctrl_r,input wire
[3:0] address,data_in,output wire
ram_cs,ram_oe,ram_we,output wire [17:0]
ram_address,inout wire [15:0] ram_data,output
wire [7:0] led_out);
wire drive_data,read_data;sram_bus_control
u(ctrl_w,ctrl_r,drive_data,read_data,ram_cs,ram_o
e,ram_we);
assign ram_address={14'b0,address};assign
ram_data=drive_data?{12'b0,data_in}:16'bz;
assign
led_out=read_data?{4'b0,ram_data[3:0]}:8'b0;endmo
dule
```

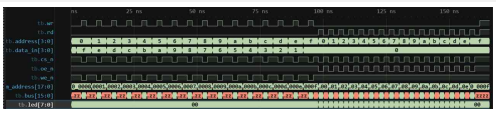


그림 13 SRAM 16주소 쓰기 및 읽기 파형



그림 14 SRAM 자기검사 PASS

10.4.2 구현 및 실제 장비 결과

초기 XDC에는 핀 지정이 없어 전체 55개 포트에서 NSTD-1과 UCIO-1 오류가 발생하였다. 교재의 SRAM 전용 핀표를 기준으로 주소 18비트, 데이터 16비트, CS/OE/WE, 입력 스위치와 LED의 PACKAGE_PIN 및 LVCMOS33을 지정하였다. 이후 write_bitstream Complete와 실패 route 0을 확인하였다. 이 회로는 외부 클록을 사용하지 않는 조합회로이므로 Timing Summary의 WNS가 inf이고 사용자 지정 타이밍 제약이 없다는 문구는 오류가 아니다.

SRAM_LED_동작확인.mp4에서는 스위치 조작에 따라 주소와 데이터 조건을 변경하고, 읽기 단계에서 LED 출력이 변화하는 실제 장비 동작을 확인하였다.

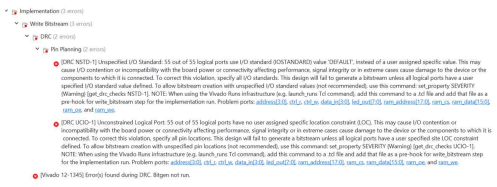


그림 15 SRAM 전체 포트 제약 누락 오류



그림 16 SRAM 비트스트림 생성 완료



그림 17 SRAM 조합회로 타이밍 요약

10.5 FPGA DAC 제어 회로

10.5.1 설계 및 시뮬레이션

8비트 ramp 레지스터는 일정한 업데이트 주기마다 1씩 증가하며 0xFF 다음에는 0x00으로 순환한다. 업데이트 시점에 DAC 데이터와 채널 선택 신호를 저장하고 write enable을 한 클록 동안 Low로 만들어 병렬 DAC에 값을 전달한다. 동일한 데이터는 LED에도 표시한다.

```
`timescale 1ns/1ps
module ramp8 #(parameter
UPDATE_CYCLES=500000)(input wire
clk_50mhz,rst_p,output reg[7:0]value,output wire
update);
reg[31:0]count;assign
update=(count==UPDATE_CYCLES-1);always@(posedge
clk_50mhz)begin if(rst_p)begin
count<=0;value<=0;end else if(update)begin
count<=0;value<=value+1'b1;end else
count<=count+1;end endmodule
module dac_parallel8(input wire
clk_50mhz,rst_p,update,channel,input
wire[7:0]value,output wire cs_n,ldac_n,output reg
we_n,a_b,output reg[7:0]data);
assign cs_n=0;assign ldac_n=0;always@(posedge
clk_50mhz)begin if(rst_p)begin
we_n<=1;a_b<=0;data<=0;end else begin
we_n<=1;if(update)begin
data<=value;a_b<=channel;we_n<=0;end end end
endmodule
module dac #(parameter
UPDATE_CYCLES=500000)(input wire
clk_50mhz,rst_p,add_sel,output wire
dac_csn,dac_ldacn,dac_wen,dac_a_b,output
wire[7:0]dac_d,led_out);
wire[7:0]value;wire update;ramp8
#(.UPDATE_CYCLES(UPDATE_CYCLES))u_ramp(clk_50mhz,
rst_p,value,update);
dac_parallel8
u_bus(clk_50mhz,rst_p,update,add_sel,value,dac_cs
n,dac_ldacn,dac_wen,dac_a_b,dac_d);assign
led_out=dac_d;endmodule
```



그림 18 DAC 데이터 증가 및 제어 파형



그림 19 DAC 자기검사 PASS

10.5.2 구현 결과와 제한사항

처음에는 clk_50mhz를 일반 I/O인 N8에 배치하여 IO pin과 BUFG 사이의 전용 클록 배치 규칙이 실패하였다. 오류 우회 설정을 사용하지 않고 메인 50 MHz 클록 핀 B6으로 수정하였다. 수정 후 WNS 14.225 ns, TNS 0.000 ns, WHS 0.191 ns, THS 0.000 ns이며 실패 endpoint와 route는 0인 상태로 비트스트림 생성을 완료하였다.

DAC의 디지털 입력 데이터, 제어 신호, 순환 동작은 시뮬레이션과 구현 결과로 검증하였다. 오실로스코프를 사용하지 않았으므로 아날로그 출력 전압과 파형에 대한 정량 측정 결과는 본 보고서에 포함하지 않았다.



그림 20 DAC 비전용 클록 핀 배치 오류

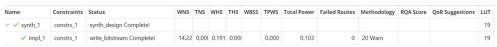


그림 21 DAC 비트스트림 생성 완료

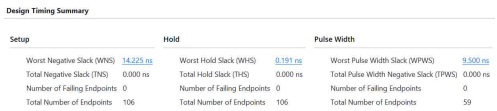


그림 22 DAC 타이밍 제약 충족

10.6 실제 영상 증거

아래 영상은 시뮬레이션 PASS와 Vivado 구현 결과가 실제 장비 동작으로 이어지는지를 확인하기 위한 기능 증거로 분석하였다. FPGA 핀은 교재의 보드 핀 구성표를 기준으로 지정하였고, UART LED 연동에서는 STM32 TX와 FPGA RX를 연결한 뒤 두 보드의 GND를 공통으로 구성하였다. Servo는 AA22, 메인 클록은 B6을 사용했으며 SRAM과 DAC의 주변회로 핀에는 LVCMOS33을 적용하였다.

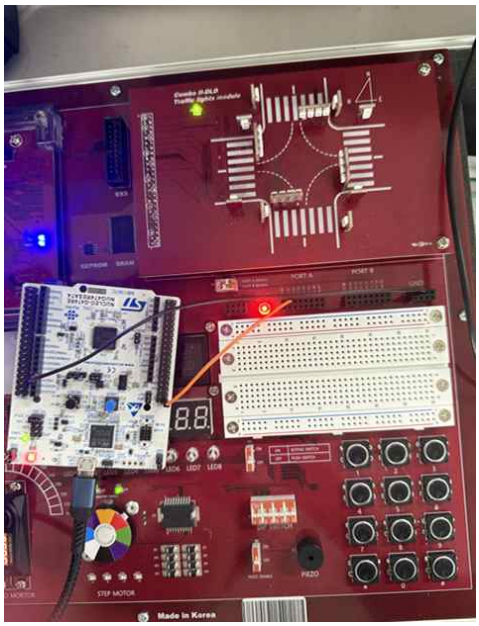


그림 23 STM32와 FPGA 실험 장비 연결 상태

10.6.1 과제 5 PC UART 에코 영상

과제5_UART_에코.mp4 에서는 VS Code Serial Monitor가 COM4에 연결된 뒤 Hello, World!가 반복 출력된다. 입력창에서 문자 1을 전송하면 Received: 1, byte: 0x31이 표시된다. 표시 문자와 ASCII 바이트 값이 일치하므로 PC에서 USB Virtual COM Port와 USART2를 거쳐 STM32의 getchar 및 printf 경로로 되돌아오는 에코 기능이 정상임을 확인하였다. 이 영상은 문자 단위 기능 검증이며 UART 비트 주기 자체를 측정한 자료는 아니다.

10.6.2 과제 6 STM32 UART 송신 영상

과제6_UART_송신.mp4 에서는 두 개의 시리얼 화면을 함께 사용하였다. 송신 측 화면에는 TX char, byte, result가 출력되고, 수신 측 화면에는 전송한 문자가 순서대로 표시된다. result: 0은 HAL_UART_Transmit가 HAL_OK를 반환했다는 뜻이고, 별도 수신 화면에 같은 문자가 나타난 사실이 외부 UART 선로를 통한 실제 수신을 뒷받침한다. 따라서 함수 호출 성공 여부뿐 아니라 송신 데이터와 수신 데이터의 대응까지 확인하였다.

10.6.3 과제 7 STM32 FPGA UART 연동 영상

과제7_FPGA_LED_연동.mp4 에서는 노트북의 Serial Monitor, STM32 보드, FPGA 실험 장비가 한 화면에 나타난다. PC에서 보낸 0과 1 명령이 STM32 USART3를 통해 FPGA UART RX로 전달되고, 명령에 따라 보드 LED 상태가 바뀌는 과정을 확인하였다. 이는 PC 입력, STM32 송신, FPGA 수신기, 명령 해석, LED 출력으로 이어지는 전체 경로의 기능 검증이다. 영상 판정에는 STM32와 FPGA 사이의 공통 GND 및 TX-RX 배선을 전제로 하였다.

10.6.4 Servo PWM 실제 동작 영상

서보_PWM_동작1.mp4에서는 보드 버튼을 누를 때 서보 혼이 기존 위치에서 다른 위치로 회전하는 모습이 확인된다. 서보_PWM_동작2.mp4에서는 입력을 반복할 때 서보가 지정 위치로 이동하고 그 위치를 유지하는 동작을 확인하였다. 두 영상은 버튼 입력에 따라 pulse_us가 단계적으로 변하고 PWM 출력이 실제 기계적 위치 변화로 이어짐을 보여 준다. 다만 1.0 ms, 1.5 ms, 2.0 ms의 정확한 펄스 폭은 오실로스코프로 측정하지 않았으므로 해당 수치는 자기검사 테스트벤치 결과에 근거한다.

10.6.5 SRAM 실제 장비 영상

SRAM_LED_동작확인.mp4에서는 FPGA 실험 장비의 DIP 스위치와 제어 입력을 조작할 때 LED 출력이 변경되는 모습을 확인하였다. 이는 주소와 입력 데이터, 쓰기 및 읽기 제어가 외부 SRAM 인터페이스와 LED 표시 경로에 반영됨을 보여 준다. 영상만으로 모든 16개 주소와 16비트 버스의 타이밍을 각각 판정하지는 않았으며, 전체 주소 순회와 버스 해제 여부는 시뮬레이션의 PASS SRAM 결과와 함께 검증하였다.

10.6.6 DAC 검증 범위

DAC는 별도의 실제 동작 영상과 오실로스코프 측정을 수행하지 않았다. 따라서 8비트 카운트 데이터 증가, 0xFF 이후 0x00 순환, 채널 선택, write 신호와 데이터 유지 조건은 자기검사 테스트벤치와 Vivado 구현 및 비트스트림 생성 결과로만 판정하였다. 아날로그 출력 전압, 선형성, 상승 시간과 파형 품질은 본 실험의 확인 범위에 포함하지 않았다.

- **과제5_UART_에코.mp4** / 핵심 관찰: 문자 1과 0x31 에코 일치 / 판정: 과제 5 정상
- **과제6_UART_송신.mp4** / 핵심 관찰: 송신 기록과 별도 수신 표시 대응 / 판정: 과제 6 정상
- **과제7_FPGA_LED_연동.mp4** / 핵심 관찰: STM32 명령에 따른 FPGA LED 변화 / 판정: 과제 7 정상
- **서보_PWM_동작1.mp4·서보_PWM_동작2.mp4** / 핵심 관찰: 버튼 입력과 서보 위치 이동·유지 / 판정: Servo 실동작
- **SRAM_LED_동작확인.mp4** / 핵심 관찰: 스위치 조건에 따른 SRAM LED 출력 변화 / 판정: SRAM 실동작

11. 실험 결과와 예상 결과 간의 비교

- **과제 5** / 예상: 입력 문자를 그대로 반환 / 실제: 1과 0x31 일치 / 해석: 정상
- **과제 6** / 예상: 송신 문자가 수신 모니터에 표시 / 실제: 문자별 result 0 및 수신 표시 / 해석: 정상
- **과제 7** / 예상: 문자 명령에 따라 FPGA LED 변경 / 실제: 송신 기록과 보드 LED를 영상 확인 / 해석: 정상
- **UART LED** / 예상: 0/1/2 명령 및 에코 / 실제: 자기검사 PASS, 타이밍 통과 / 해석: 정상
- **Servo** / 예상: 1/1.5/2 ms와 위치 변화 / 실제: 시뮬레이션 PASS, 실제 이동 / 해석: 정상
- **SRAM** / 예상: 주소별 쓰기 후 동일 값 읽기 / 실제: 16주소 PASS, LED 변화 / 해석: 정상
- **DAC** / 예상: 8비트 데이터 증가와 순환 / 실제: 260회 쓰기 PASS, 구현 성공 / 해석: 아날로그 측정 제외

12. 문제 해결 과정

- Servo: 출력 포트의 LOC와 IOSTANDARD 누락을 AA22와 LVCMOS33 제약 추가로 해결하였다.
- SRAM: 55개 포트의 핀과 I/O 표준이 모두 비어 있던 XDC를 교재 핀표에 따라 완성하였다.
- DAC: 일반 I/O N8을 50 MHz 클록으로 지정해 발생한 BUFG 배치 오류를 전용 메인 클록 B6으로 수정하였다.
- Serial Monitor: 한글 UTF-8 표시가 깨지는 경우에도 문자, 16진수 바이트, HAL 반환값으로 통신 결과를 판정하였다.

오류를 경고로 강등하거나 CLOCK_DEDICATED_ROUTE FALSE로 우회하지 않고 실제 장비 핀과 회로 의도에 맞게 수정하였다. 이 과정으로 시뮬레이션 성공과 물리 구현 가능성을 분리하여 검증할 필요가 있음을 확인하였다.

13. 종합 고찰

이번 실험에서는 MCU의 직렬 통신과 FPGA의 병렬·실시간 주변회로 제어를 하나의 검증 흐름으로 연결하였다. STM32에서는 HAL 반환값과 수신 문자를 통해 소프트웨어 통신 경로를 확인했고, FPGA에서는 테스트벤치 PASS만으로 끝내지 않고 핀 제약, 합성, 배치배선, 타이밍, 비트스트림까지 확인하였다. 특히 핀 누락과 비전용 클록 사용은 기능 시뮬레이션에서는 드러나지 않지만 실제 구현 단계에서 실패하므로, 논리 검증과 물리 제약 검증을 모두 수행해야 한다.

Servo와 SRAM은 영상에서 실제 기계적 위치 변화와 LED 출력을 확인하였다. STM32-FPGA UART 연동 역시 송신 기록과 보드 상태를 한 화면에서 검토하였다. DAC는 디지털 제어 시퀀스와 비트스트림 생성까지 검증했으나 오실로스코프를 사용하지 않았기 때문에 아날로그 출력 특성을 정량적으로 평가하지 않았다. 따라서 본 실험의 DAC 결론은 디지털 인터페이스 구현 성공 범위로 한정한다.

14. 증빙 및 source commit 링크(GitHub)

- **저장소** / 내용: https://github.com/dalkomys777/fpga_lab6
- **브랜치** / 내용: main
- **결과 확인 기준 커밋** / 내용: 44c96f0b96bd02a22aabaa21bb4b0714216e2c64
- **예비 검증 기준 커밋** / 내용: 2d2ca03526d2908695ce478989e4f41778d8edef
- **예비 검증 태그** / 내용: prelab-evidence-v1.0.0
- **최종 제출 태그** / 내용: lab6-final-v1.0.0 https://github.com/dalkomys777/fpga_lab6/releases/tag/lab6-final-v1.0.0
- **STM32** / 내용: stm32/task5_pc_echo, task6_mcu_to_fpga, task7_uart_led
- **FPGA** / 내용: circuits/00_uart_led, 01_servo, 02_sram, 03_dac

14.1 보고서·공통 자료

- 저장소: https://github.com/dalkomys777/fpga_lab6
- 결과보고서 PDF(업로드 후): https://github.com/dalkomys777/fpga_lab6/blob/main/reports/post/%EA%B2%B0%EA%B3%BC%EB%A6%AC%ED%8F%AC%ED%8A%B8_LAB6%2020254440031%EA%B9%80%EC%98%81%EC%84%9C.pdf
- 예비보고서 PDF: https://github.com/dalkomys777/fpga_lab6/blob/main/reports/pre/%EC%98%88%EB%B9%84%EB%A6%AC%ED%8F%AC%ED%8A%B8_LAB6%202025440031%EA%B9%80%EC%98%81%EC%84%9C.pdf
- 프로젝트 설명: https://github.com/dalkomys777/fpga_lab6/blob/main/README.md
- 핀 확인 안내: https://github.com/dalkomys777/fpga_lab6/blob/main/PIN_CONFIRMATION.md
- 사용자 캡처 폴더: https://github.com/dalkomys777/fpga_lab6/tree/main/evidence/user_captures
- 예비 검증 자료 폴더: https://github.com/dalkomys777/fpga_lab6/tree/main/evidence/prelab

14.2 STM32 소스

- 과제5: https://github.com/dalkomys777/fpga_lab6/blob/main/stm32/task5_pc_echo/ece2_474_routine.c
- 과제6: https://github.com/dalkomys777/fpga_lab6/blob/main/stm32/task6_mcu_to_fpga/ece2_474_routine.c
- 과제7: https://github.com/dalkomys777/fpga_lab6/blob/main/stm32/task7_uart_led/ece2_474_routine.c

14.3 FPGA 설계·테스트벤치·XDC

- UART LED 설계: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/00_uart_led/src/lab6_uart_led.v
- UART 송수신: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/00_uart_led/src/uart.v
- UART LED TB: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/00_uart_led/sim/tb_design.v
- UART LED XDC: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/00_uart_led/constraints/lab6_uart_led.xdc
- Servo 설계: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/01_servo/src/servo.v
- Servo TB: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/01_servo/sim/tb_design.v
- Servo XDC: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/01_servo/constraints/lab6_servo.xdc
- SRAM 설계: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/02_sram/src/sram.v

- SRAM TB: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/02_sram/sim/tb_design.v
- SRAM XDC: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/02_sram/constraints/lab6_sram.xdc
- DAC 설계: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/03_dac/src/dac.v
- DAC TB: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/03_dac/sim/tb_design.v
- DAC XDC: https://github.com/dalkomys777/fpga_lab6/blob/main/circuits/03_dac/constraints/lab6_dac.xdc

14.4 시뮬레이션 원본 파형

- UART LED VCD: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/prelab/00_uart_led/wave.vcd
- Servo VCD: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/prelab/01_servo/wave.vcd
- SRAM VCD: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/prelab/02_sram/wave.vcd
- DAC VCD: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/prelab/03_dac/wave.vcd

재현 방법: 저장소의 README 및 각 회로 폴더의 설계 소스, 테스트벤치와 XDC를 이용해 시뮬레이션과 구현을 다시 실행할 수 있다. 보고서 작성 시점 이후 변경된 코드와 증거 자료는 최종 제출 전에 커밋하고 태그를 확정한다.

14.5 실험 영상 링크

과제5_UART_에코.mp4: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/videos/%EA%B3%BC%EC%A0%9C5_UART_%EC%97%90%EC%BD%94.mp4

과제6_UART_송신.mp4: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/videos/%EA%B3%BC%EC%A0%9C6_UART_%EC%86%A1%EC%8B%A0.mp4

과제7_FPGA_LED_연동.mp4: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/videos/%EA%B3%BC%EC%A0%9C7_FPGA_LED_%EC%97%B0%EB%8F%99.mp4

서보_PWM_동작1.mp4: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/videos/%EC%84%9C%EB%B3%B4_PWM_%EB%8F%99%EC%9E%911.mp4

서보_PWM_동작2.mp4: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/videos/%EC%84%9C%EB%B3%B4_PWM_%EB%8F%99%EC%9E%912.mp4

SRAM_LED_동작확인.mp4: https://github.com/dalkomys777/fpga_lab6/blob/main/evidence/videos/SRAM_LED_%EB%8F%99%EC%9E%91%ED%99%95%EC%9D%B8.mp4

15. 참고문헌

[1] ece2_474/docs/STM32_2.md, '컴퓨터와 FPGA 통신', 2026-10-26 실습 안내.

[2] ece2_474/docs/FPGA_additional.md, 'Servo·SRAM·DAC 설계 모듈 및 테스트벤치'.

[3] 전자전기컴퓨터설계실험II, LAB6 강의 및 실습 자료, 서울시립대학교, 2026.

[4] STMicroelectronics, Description of STM32G4 HAL and low-layer drivers, UM2570.

[5] AMD, Vivado Design Suite User Guide: Using Constraints, UG903, AMD, 2026.

[6] AMD, Vivado Design Suite User Guide: Design Analysis and Closure Techniques, UG906, AMD, 2026.

[7] Arm, Cortex-M4 Devices Generic User Guide, exception and interrupt handling.

[8] fpga_lab6 source repository, https://github.com/dalkomys777/fpga_lab6, accessed 2026-10-28.